WUMPUS WORLD

DYNAMIC LOGIC AGENT

Propositional Logic · CNF Conversion · Resolution Refutation

STUDENT
Waleed Nadeem
Roll No: 23F-0628
Artificial Intelligence — Q6

DEPLOYED APP
dynamic-wumpus-game-agent.vercel.app
SOURCE CODE
github.com/Rao-Waleed-Nadeem/...

This report presents the implementation of a Knowledge-Based Ag agent navigates a grid of arbitrary dimensions, collecting percep Propositional Logic Knowledge Base (KB). Before entering an Resolution Refutation engine — converting the KB to Conjunctive resolving clause pairs to prove whether a cell is safe. The system a propositional logic formulas.

1 Propositional Logic & Knowledge Base

1.1 Propositional Logic Primer

Propositional Logic is a formal system in which atomic propositions are combined using logical connectives (AND, OR, NOT, IMPLIES, IFF) to form compound sentences. Each proposition has a truth value: **True** or **False**. The Wumpus World uses propositional variables to represent the state of each cell:

Variable	Meaning	Example
$P_{r,c}$	Cell (r,c) contains a Pit	$P_{1,2}$ = True means pit at row 1, col 2
$W_{r,c}$	Cell (r,c) contains the Wumpus	$W_{2,3}$ = False means no Wumpus there
$B_{r,c}$	Agent senses Breeze at (r,c)	$B_{0,0}$ = False means no adjacent pit
$S_{r,c}$	Agent senses Stench at (r,c)	$S_{1,1}$ = True means Wumpus is adjacent

1.2 Knowledge Base (KB)

The Knowledge Base is a set of logical sentences (formulas) that encode everything the agent knows about the world. The agent uses two operations:

TELL(KB, sentence)	Add a new sentence to the KB when a percept is received

ASK(KB, query)	Query whether the KB entails a sentence (KB |= query)

1.3 Percept Rules Added to KB

When the agent visits cell (r, c) and receives percepts, it TELLs the KB the following clauses. These are derived from the causal structure of the Wumpus World:

Percept	CNF Clause(s) Added	Meaning
Breeze at (r,c)	`B(r,c) -> P(n1) v P(n2) v P(n3) v P(n4)`	At least one neighbor contains a pit
No Breeze at (r,c)	`-B(r,c) -> -P(n) for each neighbor n`	No neighbor contains a pit (unit clauses)
Stench at (r,c)	`S(r,c) -> W(n1) v W(n2) v W(n3) v W(n4)`	At least one neighbor contains the Wumpus
No Stench at (r,c)	`-S(r,c) -> -W(n) for each neighbor n`	No neighbor contains the Wumpus (unit clauses)
Start cell (0,0)	`-P(0,0) ^ -W(0,0)`	Agent starts safely; pit and Wumpus cannot be at (0,0)

2 Conjunctive Normal Form (CNF)

2.1 Definition

A formula is in **Conjunctive Normal Form (CNF)** if it is a conjunction (AND) of one or more **clauses**, where each clause is a disjunction (OR) of one or more **literals**. A literal is either an atomic proposition (P) or its negation (-P).

```
CNF = Clause_1  ^  Clause_2  ^  ...  ^  Clause_n
Clause_i = (L_1  v  L_2  v  ...  v  L_k)
Literal L = P  or  -P  (atomic or negated atomic)
```

2.2 Why CNF?

CNF is the standard form required by the **Resolution rule**. Any propositional formula can be converted to an equivalent CNF formula. This makes CNF a universal representation for automated theorem provers. The resolution algorithm operates exclusively on CNF clauses.

> **KEY PROPERTY**
> Key property: CNF conversion is always possible and preserves satisfiability. The resulting CNF may be exponentially larger in the worst case, but for Wumpus World formulas (which are already near-CNF), the conversion is direct and efficient.

2.3 CNF Conversion Algorithm

The standard four-step algorithm to convert any formula to CNF is:

1. **Eliminate Biconditionals (↔)**
   Replace A ↔ B with (A → B) ∧ (B → A)
2. **Eliminate Implications (→)**
   Replace A → B with ¬A ∨ B
3. **Push Negations Inward (¬)**
   Apply De Morgan's Laws: ¬(A∧B) = ¬A∨¬B ; ¬(A∨B) = ¬A∧¬B ; ¬¬A = A
4. **Distribute OR over AND**
   Apply: A ∨ (B ∧ C) = (A ∨ B) ∧ (A ∨ C) until all clauses are flat

2.4 CNF Example from Wumpus World

Consider: the agent is at (1,1) and senses a Breeze. The neighbors of (1,1) are (0,1), (2,1), (1,0), (1,2). The implication is:

```
B(1,1)  ->  P(0,1) v P(2,1) v P(1,0) v P(1,2)
        [Step 2: Eliminate implication]
-B(1,1)  v  P(0,1)  v  P(2,1)  v  P(1,0)  v  P(1,2)
[Already a single CNF clause — no further steps needed]
```

Because the Breeze axiom is already an implication with a disjunction on the right-hand side, converting to CNF requires only Step 2 (eliminating the implication). The resulting clause is added directly to the KB. When the agent observes NO Breeze, simpler unit clauses are generated:

```
-B(1,1)  :  -P(0,1),  -P(2,1),  -P(1,0),  -P(1,2)
[4 separate unit clauses, each ruling out one cell]
```

3 Resolution Refutation Engine

3.1 The Resolution Rule

The **Resolution Rule** is a single inference rule that is both *sound* (never derives false conclusions) and *complete* (can derive any entailed conclusion) for propositional logic. It operates on two CNF clauses that share a complementary literal pair:

```
If:   C1 = (L  v  A1  v  A2  v  ...)
      C2 = (-L  v  B1  v  B2  v  ...)
Then: RESOLVENT = (A1 v A2 v ... v B1 v B2 v ...)
```

The literal L and its complement -L cancel out. The remaining literals form a new clause (the resolvent) that is a logical consequence of C1 and C2. If the resolvent is the **empty clause {}**, it represents a contradiction (False).

3.2 Tautology Elimination

A resolvent that contains both L and -L for some literal L is a **tautology** (always True) and carries no information. The engine discards such resolvents to avoid unbounded clause growth and maintain efficiency:

wumpus_engine.py — resolve()

```python
def resolve(c1, c2):
    for lit in c1:
        neg = (lit[0], not lit[1])   # negation of literal
        if neg in c2:
            resolvent = (c1 - {lit}) | (c2 - {neg})
            # Tautology check: does resolvent contain P AND -P?
            for l in resolvent:
                if (l[0], not l[1]) in resolvent:
                    return None   # tautology, discard
            return frozenset(resolvent)
    return None   # no complementary pair found
```

3.3 Refutation (Proof by Contradiction)

Refutation (also called proof by contradiction or reductio ad absurdum) is the strategy used to prove a query Q from a KB:

1. **Negate the Query**
   To prove Q, assume its negation: add {-Q} as a unit clause to KB
2. **Resolve All Pairs**
   For every pair of clauses (C1, C2), compute all possible resolvents
3. **Check for Empty Clause**
   If the empty clause {} is derived, contradiction found — Q is proven (KB |= Q)
4. **Add New Clauses**
   Add all new non-tautological resolvents to the clause set
5. **Repeat or Terminate**
   If no new clauses can be derived, Q is not entailed (KB |≠ Q)

3.4 Resolution Loop — Complete Implementation

The following is the full resolution refutation function from the Wumpus engine. It implements a complete, iterative resolution loop:

wumpus_engine.py — resolution_refutation()

```
def resolution_refutation(kb_clauses, query_lit):
    """
    Prove query_lit by refutation: add -query to KB, then resolve.
    Returns (proved: bool, inference_steps: int)
    """
    neg_query = frozenset([negate(query_lit)])   # Step 1: negate
    clause_set = set(kb_clauses) | {neg_query}   # KB + {-Q}
    steps = 0

    while True:
        new_clauses = set()
        clause_list = list(clause_set)

        # Step 2: resolve every pair
        for i in range(len(clause_list)):
            for j in range(i + 1, len(clause_list)):
                resolvent = resolve(clause_list[i], clause_list[j])
                steps += 1

                if resolvent is not None:
                    if len(resolvent) == 0:   # Step 3: empty clause
                        return True, steps    # PROVED by contradiction
                    new_clauses.add(resolvent)

        # Step 5: fixpoint check — no new information
        if new_clauses.issubset(clause_set):
            return False, steps              # cannot prove Q

        clause_set |= new_clauses            # Step 4: add new clauses
```

3.5 Safety Query: ASK(KB, safe(r,c))

To determine if cell (r, c) is safe before the agent enters it, the engine must prove **both** that there is no pit AND no Wumpus at that cell. Two separate refutation calls are made:

```
safe(r,c)  iff  KB |= -P(r,c)  ^  KB |= -W(r,c)

Prove -P(r,c):  add {P(r,c)} to KB, resolve until {} derived
Prove -W(r,c):  add {W(r,c)} to KB, resolve until {} derived
```

wumpus_engine.py — KnowledgeBase.ask_safe()

```python
def ask_safe(self, r, c):
    no_pit_query    = (f"P_{r}_{c}", False)  # literal: -P(r,c)
    no_wumpus_query = (f"W_{r}_{c}", False)  # literal: -W(r,c)

    proved_no_pit,    s1 = resolution_refutation(self.clauses, no_pit_query)
    proved_no_wumpus, s2 = resolution_refutation(self.clauses, no_wumpus_query)

    total = s1 + s2
    self.total_inference_steps += total
    return (proved_no_pit and proved_no_wumpus), total
```

4 Worked Example: Step-by-Step Resolution

Scenario Setup

Agent is at (0,0). Neighbors are (0,1) and (1,0). Agent senses NO Breeze at (0,0). The KB now contains:

Clause	Source	Type
{ -P(0,0) }	Structural axiom	Unit — no pit at start
{ -W(0,0) }	Structural axiom	Unit — no Wumpus at start
{ -P(0,1) }	No Breeze at (0,0)	Unit — no pit at (0,1)
{ -P(1,0) }	No Breeze at (0,0)	Unit — no pit at (1,0)
{ -W(0,1) }	No Stench at (0,0)	Unit — no Wumpus at (0,1)
{ -W(1,0) }	No Stench at (0,0)	Unit — no Wumpus at (1,0)

Query: Is cell (1,0) safe?

To prove -P(1,0): negate the query to get {P(1,0)} and add to KB.

```
Augmented KB:  { -P(1,0) }  and  { P(1,0) } (negated query)

Resolve { -P(1,0) } with { P(1,0) }:
Complementary pair: P(1,0) and -P(1,0)
Resolvent = {} (empty clause) — CONTRADICTION!

Therefore: KB |= -P(1,0)  [No pit at (1,0) proven in 1 step]
```

Similarly, resolve { -W(1,0) } with { W(1,0) } to prove -W(1,0) in 1 step. Since both -P(1,0) and -W(1,0) are proven, the agent concludes: **cell (1,0) is SAFE**. It is marked green in the UI and added to the frontier.

5 System Architecture & Implementation

5.1 Component Overview

Component	File	Responsibility
Game Engine	wumpus_engine.py	World generation, agent actions, percept delivery
Knowledge Base	wumpus_engine.py	CNF clause storage, TELL/ASK operations
Resolution Engine	wumpus_engine.py	CNF resolution loop, tautology filter, step counter
REST API	app.py	Flask server exposing engine to frontend via HTTP
Web UI	index.html	Grid visualization, metrics dashboard, controls

5.2 Agent Decision Policy (Auto Play)

When in autonomous mode, the agent follows a priority-ordered decision policy:

Priority	Action	Condition
1 (Highest)	Move to unvisited safe neighbor	cell_status == SAFE and not visited
2	BFS to nearest safe frontier	No direct safe neighbor; backtrack via visited cells
3 (Lowest)	Climb out of cave	Has gold, or no safe moves exist

6 Real-Time Metrics & Complexity

6.1 Metrics Tracked

Metric	Description
Inference Steps	Total resolution operations performed across all ASK queries
KB Clauses	Number of CNF clauses currently in the Knowledge Base
Safe Cells Found	Cells proven safe (green) + cells visited
Steps Taken	Total agent moves made in the current episode
Score	-1/step, -10/arrow, +1000/gold, +500/kill, +500/exit-with-gold

6.2 Complexity Analysis

Resolution refutation is **complete** for propositional logic but has exponential worst-case complexity $O(2^n)$ where n is the number of variables. In practice, for the Wumpus World, the KB remains small (bounded by grid size × 2 variables per cell) and inference is fast. Key observations:

- Each visited cell adds at most 6 clauses (2 unit structural + 4 neighbor implications)
- Unit propagation (unit clauses) resolves in O(1) per pair — the common case in Wumpus World
- The resolution loop terminates when no new clauses are generated (fixpoint)
- Tautology elimination prevents clause explosion and keeps the set polynomial in practice

7 Deployment & Source Code

Resource	URL
Live Demo (Vercel)	https://dynamic-wumpus-game-agent.vercel.app/
GitHub Repository	https://github.com/Rao-Waleed-Nadeem/Dynamic-Wumpus-Game-Agent

DEPLOYMENT NOTE
The live deployment uses a pure-JavaScript frontend that bundles the Python inference logic into a WebAssembly-compatible implementation for browser execution. The full Python source with Flask backend is available in the GitHub repository.

References

[1] Russell, S. & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. Chapter 7: Logical Agents; Chapter 8: First-Order Logic.

[2] Robinson, J. A. (1965). A Machine-Oriented Logic Based on the Resolution Principle. *Journal of the ACM*, 12(1), 23–41.

[3] Davis, M., Logemann, G., & Loveland, D. (1962). A machine program for theorem-proving. *Communications of the ACM*, 5(7), 394–397.

[4] Tseitin, G. S. (1968). On the complexity of derivation in propositional calculus. In *Automation of Reasoning*, Springer.